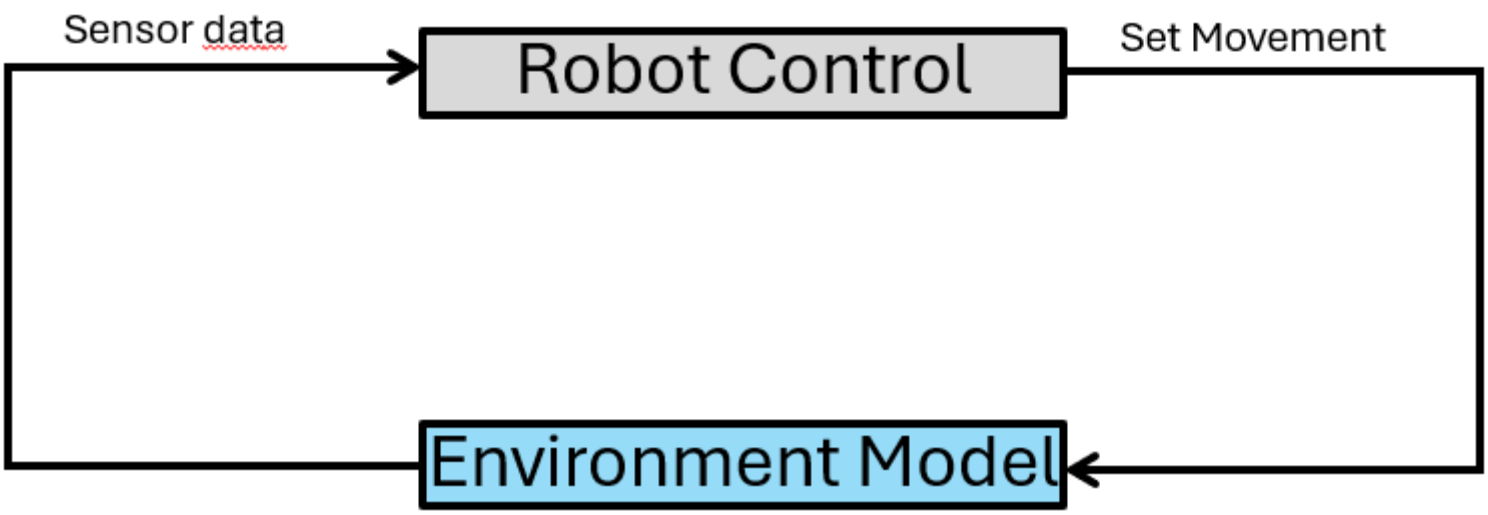
Table of Contents

Intro 1
Environment Model 1
Assumptions and System Description for the Mathematical Modeling 2
Robot Contour and Wire Signal Sensor Positions 3
Garden Definition 4
Equations of Motion and Simulation in MATLAB 5
Polyshape Functionality: inPolygon 7
Robot Control 8
Simulation 10
Introduction to Simulink 10
Time step sizes 10
Method for Logging Signals in Simulink 11
Controlling Simulink from MATLAB 11
Preload (MATLAB workspace) 11
Simulation in Simulink 11
Postprocessing in MATLAB 11
Example: Calling Simulink from MATLAB 11


```
% Matlab clearen zum Beginn:
clear
close all
```

Intro

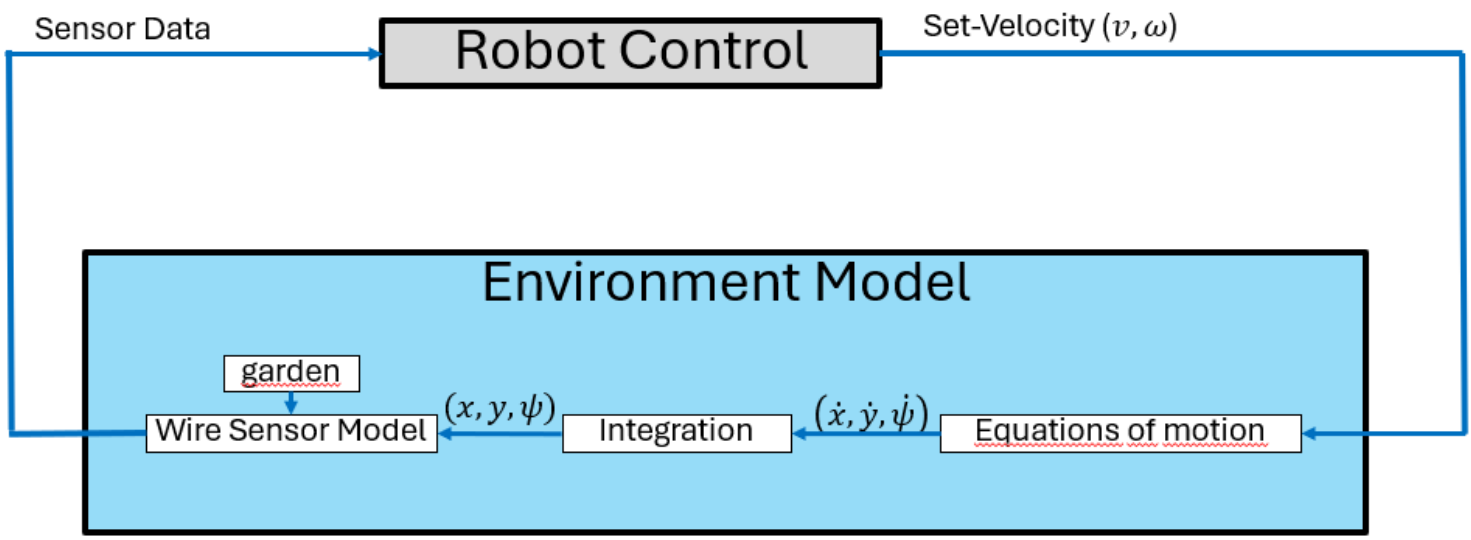
The interaction of the robot with the environment can be divided as follows. The robot control processes sensor data (wire signal coils are inside or outside the field) and issues target motions to the drive motors (e.g., driving straight or turning maneuvers). The target motions result in movements and are represented in the environment model. The mower's changed position, in turn, leads to changes in the sensor data, which the robot control processes in the next time step.



Environment Model

In the environment model, the movement of the device is modeled based on the commands from the robot control. The wire sensor data are generated from the positions of the wire sensors and serve as the input signal to the robot control.

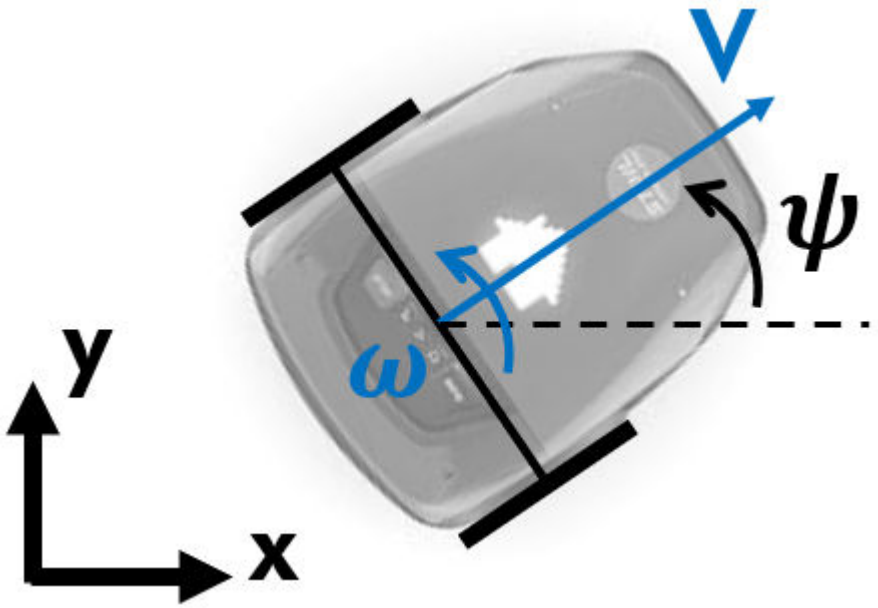
The signal chain within the environment model can be represented as follows:



Assumptions and System Description for the Mathematical Modeling

The global coordinate system of the two-dimensional plane is defined by X, Y and the rotation ψ .

The robot's locomotion is described by the longitudinal driving speed V and the rotation rate ω about the axis between the two drive wheels. The robot control specifies the desired movement via V and the rotation rate ω .



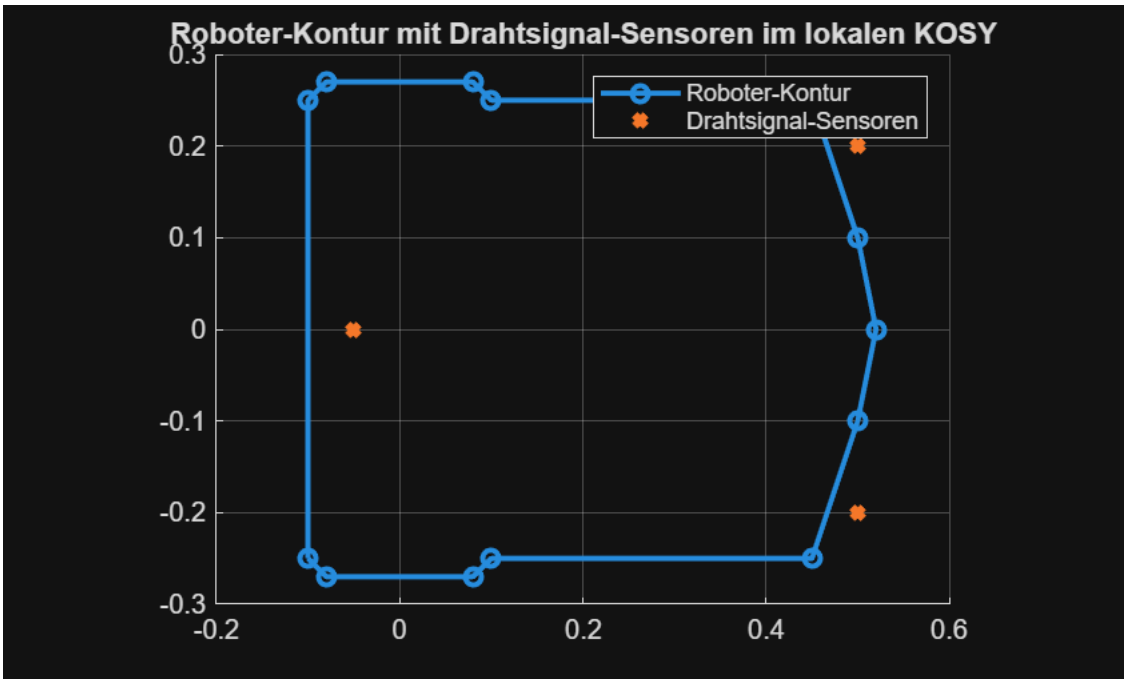
Transformed, we get this velocities in the global coordinates

$$\dot{x}(t) = v(t)\ \cos(\psi\ (t))$$
$$\dot{y}(t) = v(t)\ \sin(\psi\ (t))$$
$$\dot{\psi}\ (t) = \omega\ (t)$$

Robot Contour and Wire Signal Sensor Positions

The robot contour and the wire signal sensor positions are defined in the robot's local coordinate system.

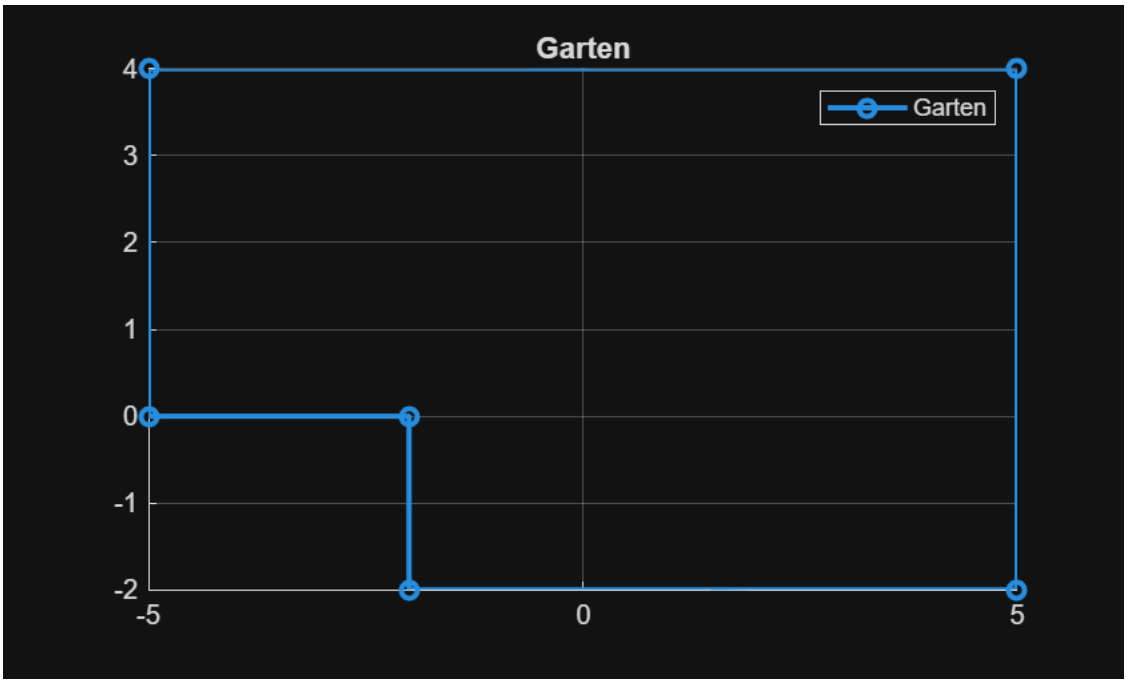
```
% Mäher-Kontur, definiert im lokalen Koordinatensystem des Roboters:
xMower_local = [-0.1 -0.08 0.08 0.1 0.45 0.5 0.52 0.5 0.45 0.1 0.08 -0.08 -0.1
-0.1];
yMower_local = [-0.25 -0.27 -0.27 -0.25 -0.25 -0.1 0 0.1 0.25 0.25 0.27 0.27
0.25 -0.25];

% Drahtsignalsensor-Positionen, definiert im lokalen Koordinatensystem des
Roboters:
xSpulen_local = [0.5 0.5 -0.05];
ySpulen_local = [0.2 -0.2 0];

% Plot Mäher und Spulen im lokalen KOSY
clf
hold on, grid on
daspect([1 1 1])
title('Roboter-Kontur mit Drahtsignal-Sensoren im lokalen KOSY')
plot(xMower_local,yMower_local,'-o','LineWidth',2,'DisplayName','Roboter-Kontur')
plot(xSpulen_local,ySpulen_local,'x','LineWidth',3,'DisplayName','Drahtsignal-
Sensoren')
legend()
hold off
```



Garden Definition

The garden, i.e., the course of the boundary wire, is defined in the global coordinate system.

```
% Garten-Kontur
garden_x = [-2,5,5,-5,-5,-2,-2];
garden_y = [-2,-2,4,4,0,0,-2];
clf
hold on, grid on
daspect([1 1 1])
title('Garten')
plot(garden_x,garden_y,'-o','LineWidth',2,'DisplayName','Garten')
legend()
hold off
```

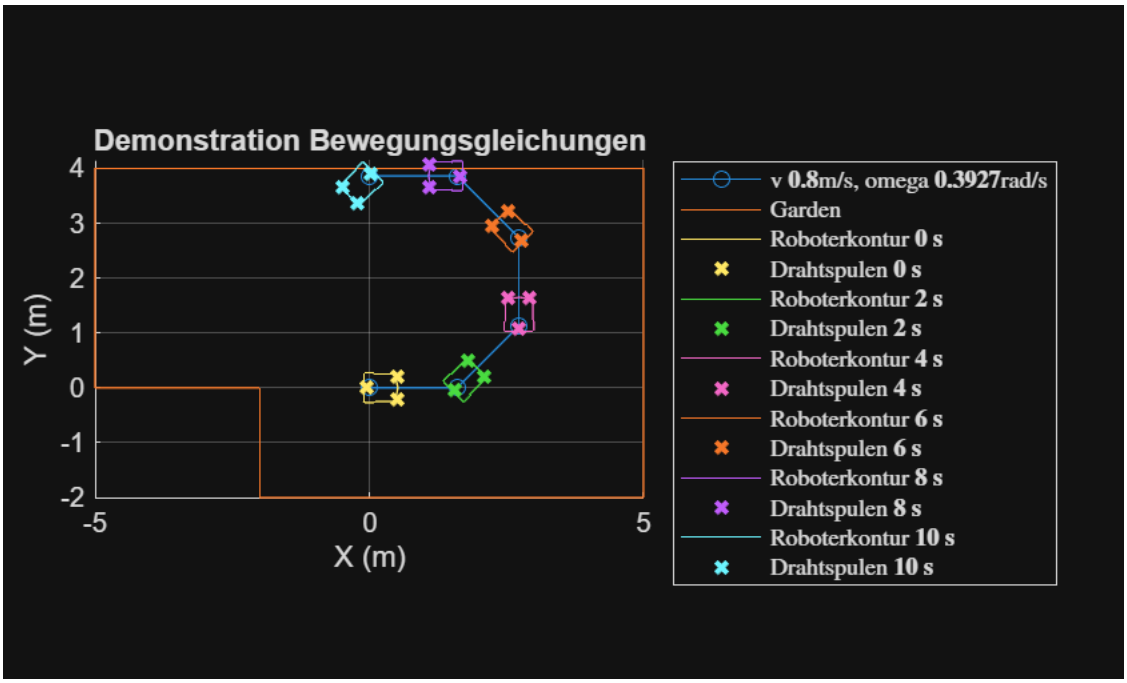


Equations of Motion and Simulation in MATLAB

The robot with its sensors is, by way of example, moved along a circular arc in the defined garden. The equations of motion are integrated in MATLAB, and the coordinate transformation from local to global is applied.

```
% Betrachteter Zeitraum: 0 bis 10 s, diskretisiert in 2 s -Schritten:
t = 0:2:10;
piinkr = pi/40;
pidefault = piinkr*4*2;
pimax = pi/2;
pimin = -pi/2;

% Längsgeschwindigkeit v und Drehrate omega des Roboters:
v_const = 0.8; % in m/s
omega_const = 0.392699081698724; % in rad/s. Mathematisch positiv gegen den UZS.

% v und omega diskretisiert in jedem Zeitschritt. v und omega sind konstant
% über die Zeit:
v = ones(length(t),1) * v_const;
omega = ones(length(t),1) * omega_const;

% Initialzustand des Roboters. Position und Orientierung:
x(1) = 0;
y(1) = 0;
psi(1) = 0;

for i_t = 1:length(t)
% Aufstellen der Bewegungsgleichung:
x_dot(i_t) = v(i_t) * cos(psi(i_t));
y_dot(i_t) = v(i_t) * sin(psi(i_t));
psi_dot(i_t) = omega(i_t);
```

```
% Rotations-Matrix für KOSY-Trafo lokal --> global
rotMat = [cos(psi(i_t)) -sin(psi(i_t)); sin(psi(i_t)) cos(psi(i_t))];
xMower(i_t,:) = x(i_t) + rotMat(1,:) * [xMower_local; yMower_local];
yMower(i_t,:) = y(i_t) + rotMat(2,:) * [xMower_local; yMower_local];
xSpulen(i_t,:) = x(i_t) + rotMat(1,:) * [xSpulen_local; ySpulen_local];
ySpulen(i_t,:) = y(i_t) + rotMat(2,:) * [xSpulen_local; ySpulen_local];

% Diskrete Integration: Berechnung des nächsten Zeitschritts
if i_t == length(t) %nächster Zeitschritt nicht mehr benötigt
    break
end
delta_t = t(i_t+1) - t(i_t);
x(i_t+1) = x(i_t) + x_dot(i_t) * delta_t;
y(i_t+1) = y(i_t) + y_dot(i_t) * delta_t;
% y = ode45( lsim
psi(i_t+1) = psi(i_t) + psi_dot(i_t) * delta_t;
end

clf
title('Demonstration Bewegungsgleichungen')
xlabel('X (m)')
% xlim([0 10])
% ylim([-2 2])
ylabel('Y (m)')
hold on, grid on
daspect([1 1 1])
plot(x,y,'-o','DisplayName',['v \textbf{' num2str(v_const) '} m/s, omega
\textbf{' num2str(omega_const) '} rad/s'])
plot(garden_x,garden_y,'DisplayName','Garden')

for i_t = 1:length(t)
hh = plot(xMower(i_t,:),yMower(i_t,:),'-','DisplayName',['Roboterkontur
\textbf{' num2str(t(i_t)) ' s}']);
plot(xSpulen(i_t,:),ySpulen(i_t,:),'x','LineWidth',2,'Color',hh.Color,'DisplayNam
e',['Drahtspulen \textbf{' num2str(t(i_t)) ' s}'])
end
legend('Interpreter','latex','location','eastoutside')
```

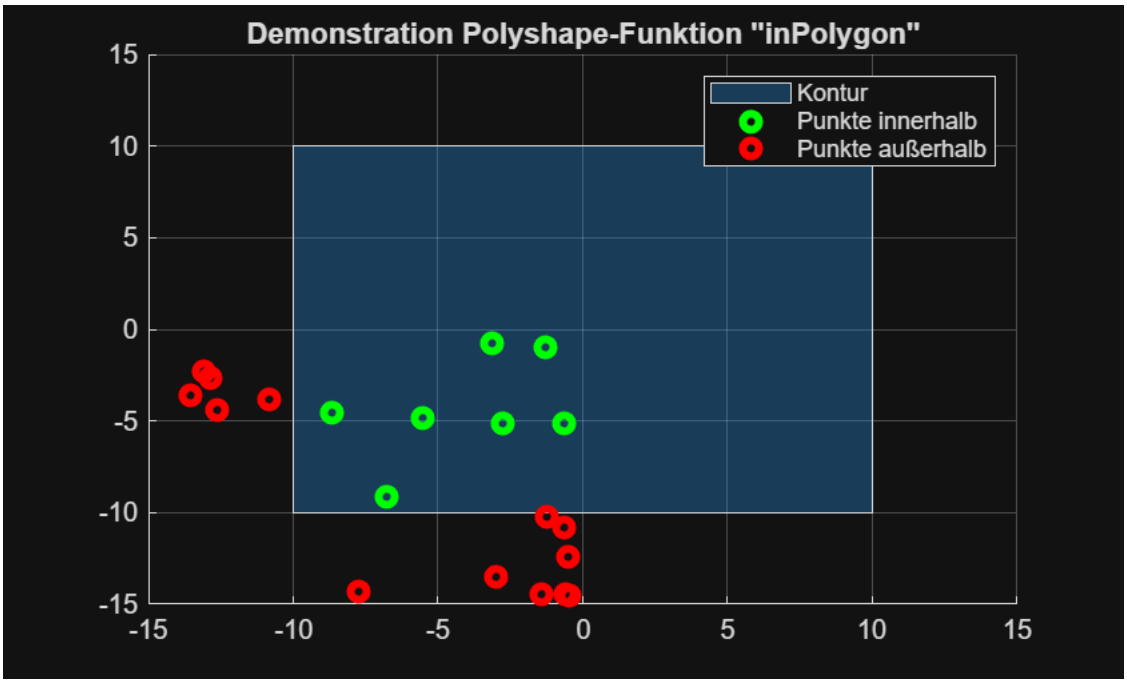


Polyshape Functionality: inPolygon

If the coil position is inside the mowing area, the respective coil signal is "inside", otherwise "outside" the mowing area.

The inpolygon functionality could be helpful. For example, a contour is defined and a point cloud. If the points are inside the contour, they should be plotted green; otherwise red.

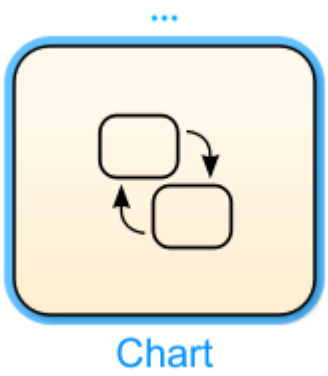
```
clf
konturX = [-10,10,10,-10];
konturY = [-10,-10,10,10];

punkte_X = rand(20,1)*15-15;
punkte_Y = rand(20,1)*15-15;

title('Demonstration Polyshape-Funktion "inPolygon"')
hold on, grid on
plot(polyshape(konturX,konturY),'DisplayName','Kontur')

i_inside = find(inpolygon(punkte_X,punkte_Y,konturX,konturY));
i_outside = find(~inpolygon(punkte_X,punkte_Y,konturX,konturY));
plot(punkte_X(i_inside),punkte_Y(i_inside),'o','LineWidth',3,'Color',[0 1
0],'DisplayName','Punkte innerhalb')
plot(punkte_X(i_outside),punkte_Y(i_outside),'o','LineWidth',3,'Color',[1 0
0],'DisplayName','Punkte außerhalb')

legend()
```



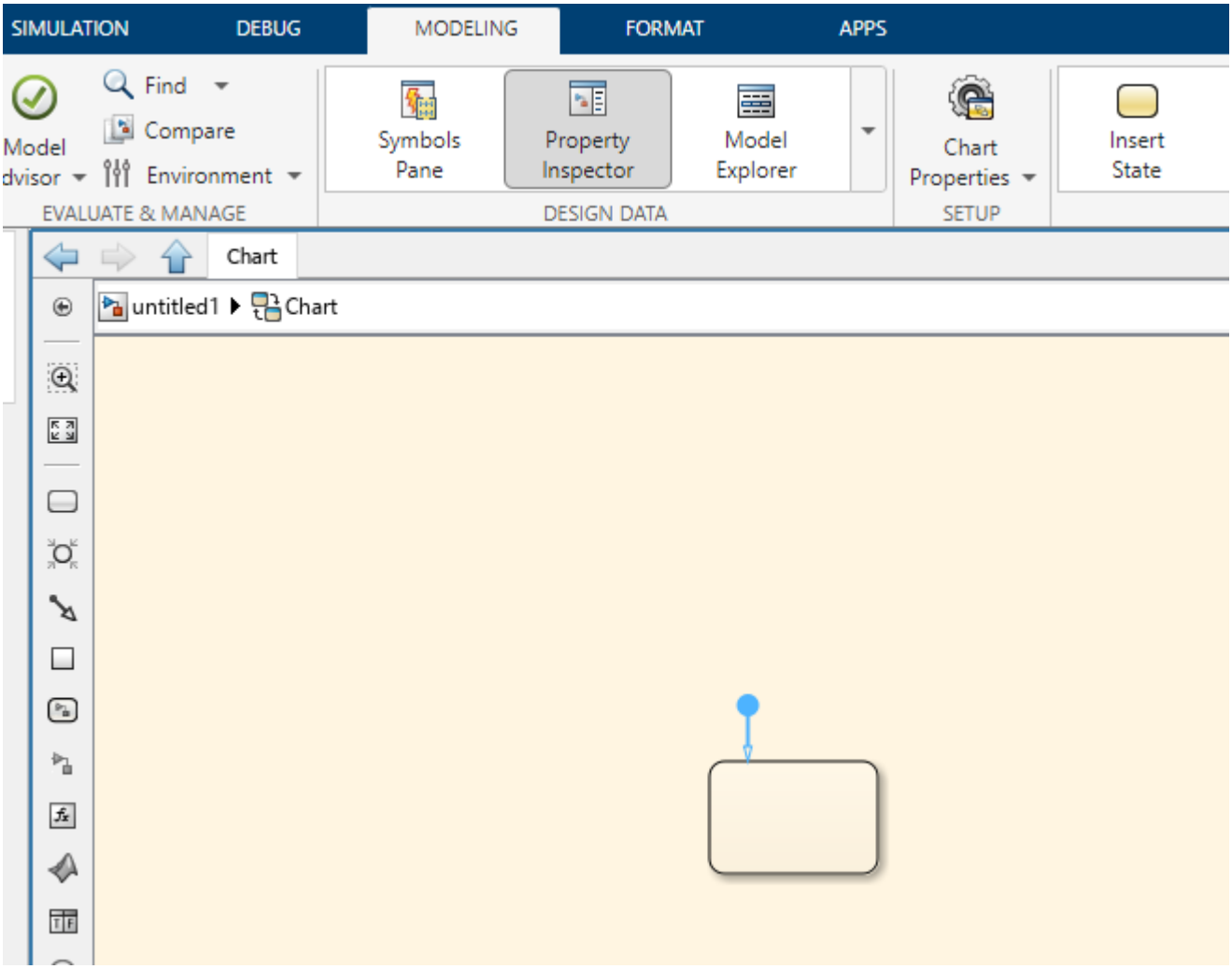
Robot Control

State machines can be represented in `Stateflow` (part of `Simulink`). `Simulink` can be opened from the MATLAB command window using the command "`simulink`". An introduction to `Simulink` will be given later.

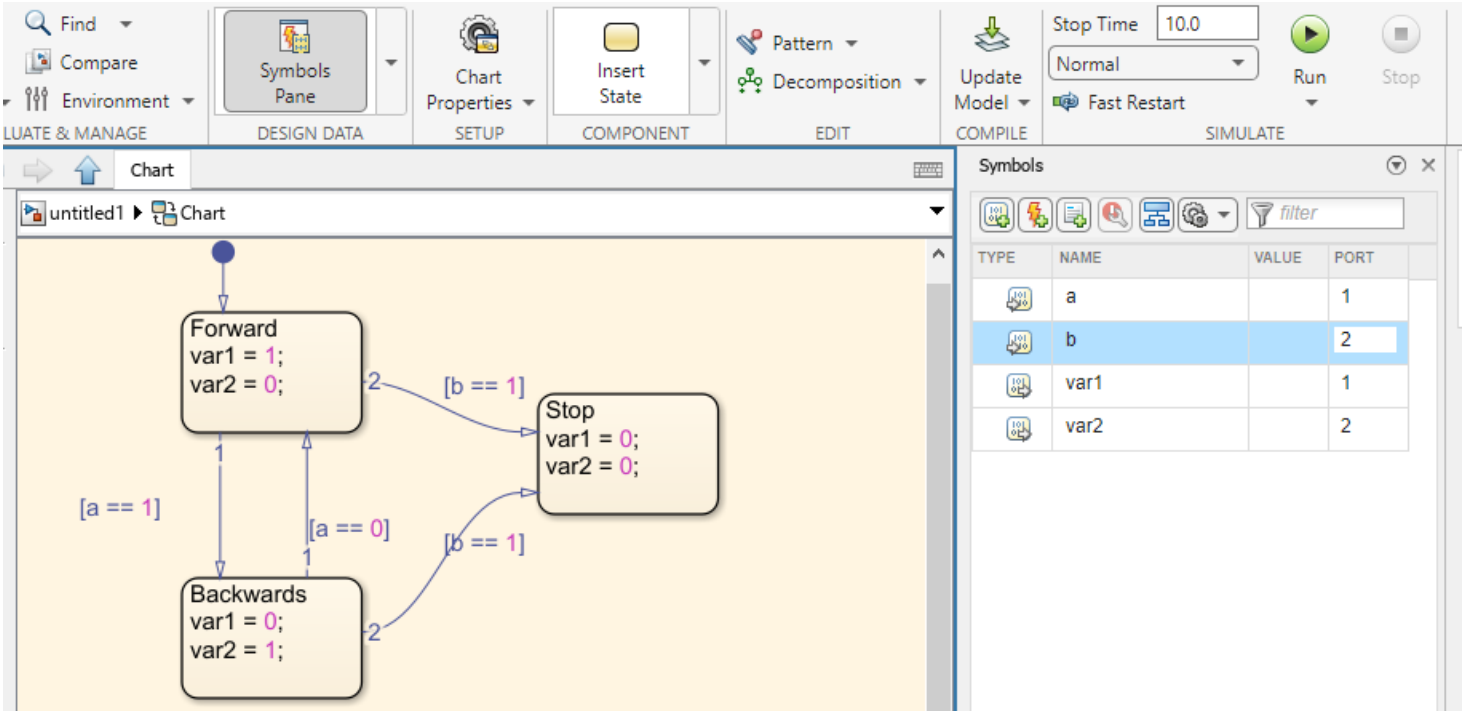
By double-clicking in the `Simulink` window and entering "stateflow", a `Stateflow` chart can be added:



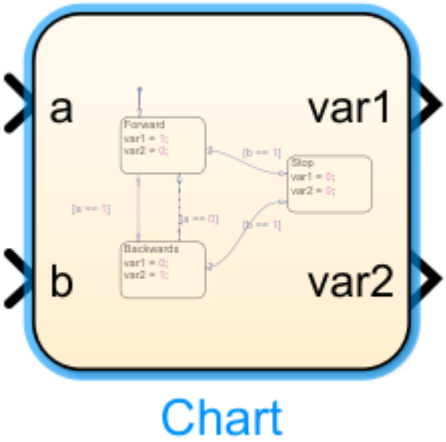
States can be added vie the tab Modeling



Different states are connected by transitions (conditions). Outputs are set within the states and inputs are used in the conditions. The Symbols pane is used to define the variables (inputs, outputs, local variables).



The inputs and outputs of the Stateflow chart are then visible in Simulink:



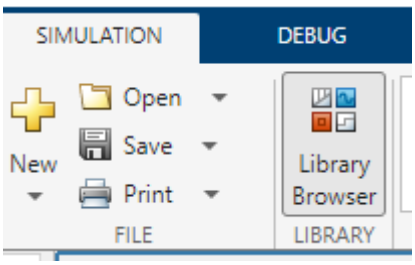
Simulation

First a brief overview of Simulink is given. To be able to evaluate the simulation afterwards, a method for signal logging in Simulink is presented.

Introduction to Simulink

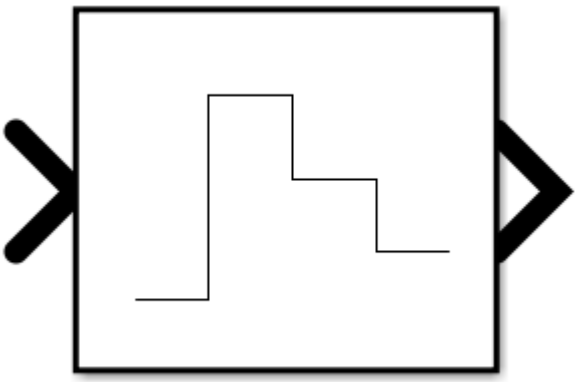
Simulink is an extension of MATLAB and is ideal for building simulation environments.

Signal lines connect individual blocks (for example, Stateflow, MATLAB Function blocks, mathematical operations, …). Double-clicking the Simulink background area lets you search for and add blocks. You can also find blocks via the Library Browser, for example.



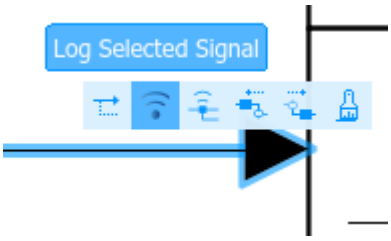
Time step sizes

By default, Simulink uses a variable time-step for simulation. This is advantageous because it allows finer discretization at times with relevant changes (for example, during collisions). However, the robot controller should operate (as on the real robot) on a fixed 10 ms cycle. To account for this, the interfaces to and from the robot controller must each be equipped with Zero-Order-Hold blocks. The sample time is specified in that block.



Method for Logging Signals in Simulink

By clicking a signal arrow, a menu appears. Signal logging can be enabled there. The logged signals should be named by double-clicking the signal arrow.



After running the simulation in Simulink, the logged signals are accessible in the MATLAB workspace under "`out`".

Controlling Simulink from MATLAB

MATLAB (analyses, variable definitions) and Simulink (simulation model) complement each other very well.

Preload (MATLAB workspace)

By default, Simulink can access variables in the MATLAB workspace. It is therefore advisable to define variables via a MATLAB script and load them into the workspace before running the Simulink model.

Simulation in Simulink

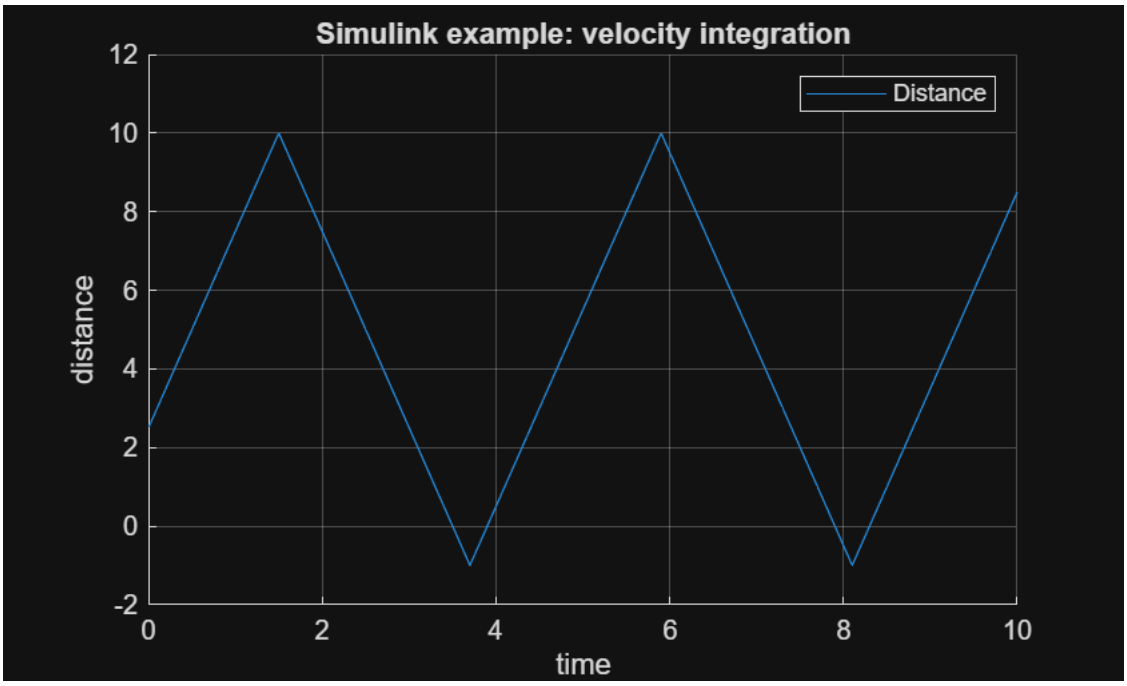
The simulation is started with `Run` and runs until the set simulation duration, i.e., the `Stop Time`.

Postprocessing in MATLAB

Logged signals are stored after the simulation in the MATLAB workspace under "`out.logsout`". Analyses and plots can be created using MATLAB scripts.

Example: Calling Simulink from MATLAB

A Simulink simulation can also be executed by calling it from MATLAB. Thus the complete chain (preload, simulation, postprocessing) can be handled in a single MATLAB script.

```
% Pre load:
velocity = 5;
distance0 = 2.5;

% Simulation:
```

```
out = sim("SimulinkExampleVelocity");

% Post processing with simulation data (out):
clf
title('Simulink example: velocity integration')
hold on, grid on
plot(out.logsout.getElement('distance').Values.Time,...
    out.logsout.getElement('distance').Values.Data,'DisplayName','Distance')
hold off
% Hinweis: Aufgrund der Abtastzeitpunkte wird geringfügig in den negativen
% Bereich gefahren
xlabel('time')
ylabel('distance')
legend()
```